DATA STRUCTURE

STACK:-

Q1. Simple stack

```
#include <iostream>
using namespace std;

class Stack
{
   int A[10], top;

public:
   Stack()
   {
      top = -1;
   }

   void push(int x)
   {
      if (top == 9)
         cout << "Stack Overflow";
```

```
    else
    {
        top++;
        A[top] = x;
    }
}

void pop()
{
    if (top == -1)
        cout << "Stack Underflow";
    else
    {
        cout << "Deleted element: " << A[top];
        top--;
    }
}

void display()
{
    for (int i = top; i >= 0; i--)
```

```
            cout << A[i] << endl;
        }
};

int main()
{
    Stack s;

    s.push(10);
    s.push(20);
    s.push(30);

    cout << "Stack elements:" << endl;
    s.display();

    s.pop();

    cout << endl << "After deletion:" << endl;
    s.display();

    return 0;
```

}

Output:

```
Stack elements:
30
20
10
Deleted element: 30
After deletion:
20
10
```

MULTIPLE STAC

```
#include <iostream>

using namespace std;


class MultipleStack

{

   int A[10], top1, top2;


public:

   MultipleStack()

   {

      top1 = -1;

      top2 = 10;

   }


   void push1(int x)
```

```
{
   if (top1 + 1 == top2)
      cout << "Stack Overflow";
   else
      A[++top1] = x;
}

void push2(int x)
{
   if (top1 + 1 == top2)
      cout << "Stack Overflow";
   else
      A[--top2] = x;
}

void pop1()
{
   if (top1 == -1)
      cout << "Stack 1 Underflow";
   else
      top1--;
}
```

```
    void pop2()
    {
        if (top2 == 10)
            cout << "Stack 2 Underflow";
        else
            top2++;
    }

    void display()
    {
        cout << "\nStack 1: ";
        for (int i = top1; i >= 0; i--)
            cout << A[i] << " ";

        cout << "\nStack 2: ";
        for (int i = top2; i < 10; i++)
            cout << A[i] << " ";
    }
};

int main()
{
    MultipleStack s;
```

```
    s.push1(10);
    s.push1(20);

    s.push2(30);
    s.push2(40);

    s.display();

    return 0;
}
```

Output:

```
Stack 1: 20 10
Stack 2: 40 30
```